ACICP302ICS

Module 1 Practical Lab Assessment Brief

Source-Code-Guided OT Simulation Lab

Document detail	Information
Module	Module 1 Integrated ICS Network Security & Attacks: Analyze, Test & Recover the Supplied OT Simulation
Activity title	Analyze, Test & Defend the Supplied Modbus/TCP OT Simulation
Assessment workflow	Governed Practical Laboratories evidence workflow
Maximum score / pass	100 marks / 50% practical pass threshold
Contribution category	Practical Laboratories
Submission mode	Private learner GitHub repository + associated evidence

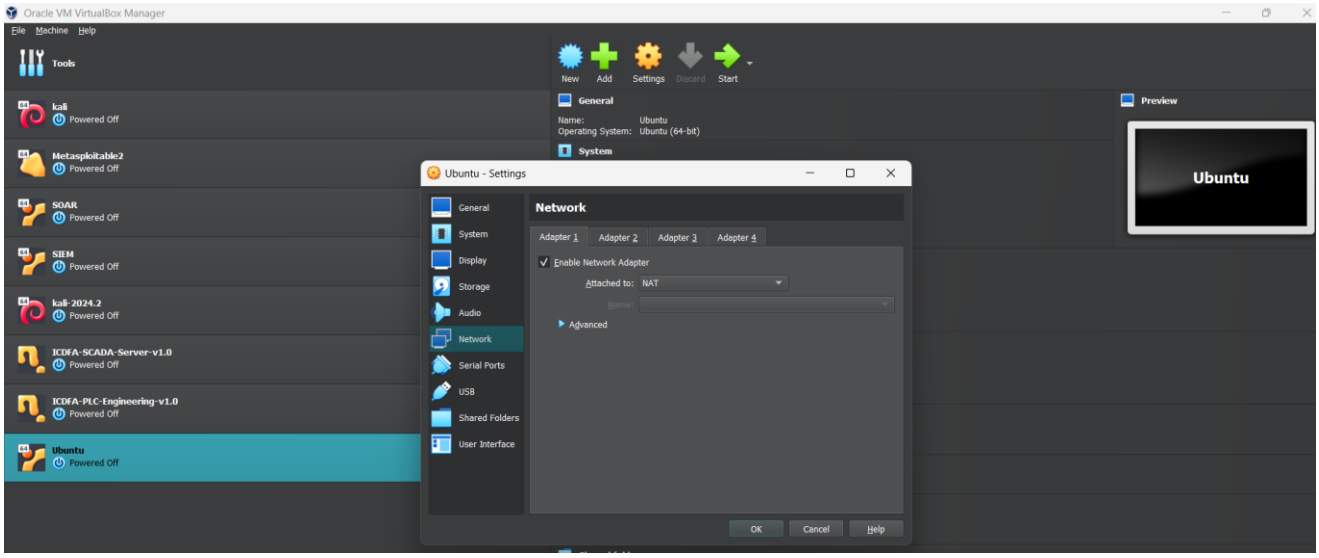
Read this first

This is a source-code-guided practical. Students use and analyze the supplied loopback-safe simulation; they are not required to build a replacement simulator. Use only the canonical student source and follow the safety boundary in this document.

International Cybersecurity and Digital Forensics Academy (ICDFA) | Version 2.0 | 05 October 2026

1. Required activities and evidence

Activity 00 - Lab preflight and safety




```
Cloning into 'ACICP302ICS-Network-Security-and-Attacks'...
remote: Enumerating objects: 130, done.
remote: Counting objects: 100% (130/130), done.
remote: Compressing objects: 100% (113/113), done.
remote: Total 130 (delta 24), reused 112 (delta 8), pack-reused 0 (from 0)
Receiving objects: 100% (130/130), 80.87 KiB | 53.00 KiB/s, done.
Resolving deltas: 100% (24/24), done.
godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks$ ls
ASSESSMENT.md          docs          PROJECT_CHALLENGE.md     starter
CONTRIBUTING.md        labs          README.md                submissions
COURSE_INFORMATION.md  NOTICE.md     SECURITY_AND_SAFETY.md
godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks$
```

```
godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks$ git rev-pa
rse HEAD
2a66f9bcf253eb3fe31c68ad52794cd535421da8
godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks$ cd ~/cours
e/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source
godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks/labs/ot-sec
urity/student-lab-source$ grep -R -nE '192\.168\.2\.139|0\.0\.0\.0' plant_sim |
| true
godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks/labs/ot-sec
urity/student-lab-source$ grep -R -n '127\.0\.0\.1' plant_sim
plant_sim/guard.py:19:PLANT_HOST = "127.0.0.1"
plant_sim/guard.py:21:HOST = "127.0.0.1"
plant_sim/plant.py:26:HOST = "127.0.0.1"
plant_sim/replay_attack.py:5:HOST = "127.0.0.1"
plant_sim/false_data_attack.py:5:HOST = "127.0.0.1"
plant_sim/dashboard.py:6:TARGET_HOST = "127.0.0.1"
plant_sim/dashboard.py:224:    app.run(host="127.0.0.1",port=8080)
plant_sim/recon_scan.py:5:HOST = "127.0.0.1"
plant_sim/dos_attack.py:6:HOST = "127.0.0.1"
plant_sim/attack_hold.py:5:HOST = "127.0.0.1"
plant_sim/dashboard_guarded.py:8:TARGET_HOST = "127.0.0.1"
plant_sim/dashboard_guarded.py:265:    app.run(host="127.0.0.1",port=8081)
plant_sim/slow_drift_attack.py:5:HOST = "127.0.0.1"
godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks/labs/ot-sec
urity/student-lab-source$
```

Activity 01 - Architecture and plant baseline

Installing Dependencies

```
godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks$ cd ~/cours
e/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source

python3 --version
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
Python 3.10.12
Requirement already satisfied: pip in ./.venv/lib/python3.10/site-packages (22.
0.2)
Collecting pip
  Downloading pip-26.2.1-py3-none-any.whl (1.8 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 447.9 kB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 22.0.2
    Uninstalling pip-22.0.2:
      Successfully uninstalled pip-22.0.2
Successfully installed pip-26.2.1
Collecting pymodbus==3.6.9 (from -r requirements.txt (line 1))
```

Starting the simulation

```
(.venv) godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks/lab
s/ot-security/student-lab-source$ cd ~/course/ACICP302ICS-Network-Security-and-
Attacks/labs/ot-security/student-lab-source
source .venv/bin/activate
cd plant_sim
(.venv) godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks/lab
s/ot-security/student-lab-source/plant_sim$
```

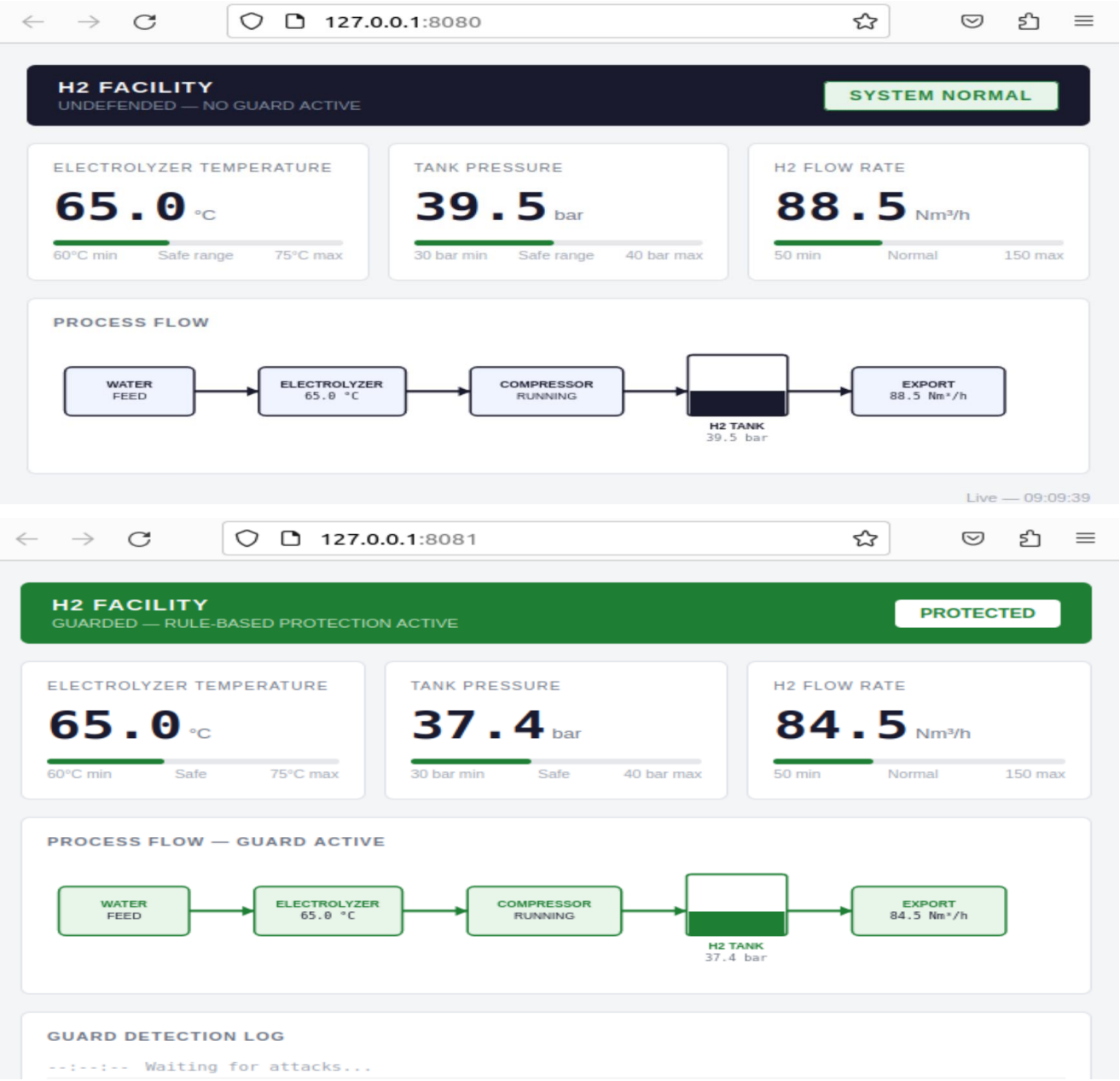
```
(.venv) godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks/lab
s/ot-security/student-lab-source/plant_sim$ python3 plant.py
2026-10-10 08:58:03,932 INFO telemetry temp=67.1C pressure=34.2bar flow=89.6Nm3
/h alarm=0
2026-10-10 08:58:03,932 INFO Plant simulator listening on 127.0.0.1:5020 (no pr
otection layer active)
2026-10-10 08:58:03,935 INFO Server listening.
2026-10-10 08:58:04,934 INFO telemetry temp=67.0C pressure=34.4bar flow=90.1Nm3
```

```
(.venv) godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks/lab
s/ot-security/student-lab-source/plant_sim$ python3 guard.py
2026-10-10 09:05:00,189 INFO Guard listening on 127.0.0.1:5021 — protecting pla
nt at 127.0.0.1:5020
2026-10-10 09:05:00,191 INFO Server listening.
```

```
(.venv) godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks/lab
s/ot-security/student-lab-source/plant_sim$ python3 dashboard.py
 * Serving Flask app 'dashboard'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment
. Use a production WSGI server instead.
 * Running on http://127.0.0.1:8080
Press CTRL+C to quit
```

```
(.venv) godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks/lab
s/ot-security/student-lab-source/plant_sim$ python3 dashboard_guarded.py
 * Serving Flask app 'dashboard_guarded'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment
. Use a production WSGI server instead.
 * Running on http://127.0.0.1:8081
Press CTRL+C to quit
```

Baseline dashboard observations



Normal-operation observations: The supplied plant simulation was started before the guard and dashboards. Both dashboard interfaces were opened inside the isolated Ubuntu VM. The initial readings were recorded before any test demonstration.

Answer to the architecture question: The simulated plant models the process. Modbus/TCP provides communication with the process data. The direct path connects to the plant on port 5020, while the guarded path uses port 5021 and applies validation before forwarding accepted writes. The dashboards provide views of the process. Confirm the exact data flow by inspecting the source code.

Activity 02 - Modbus/TCP data model and normal operations

Register map

Data point	Observed value	Unit	Displayed minimum	Displayed maximum
Electrolyzer temperature	65.0	°C	60	75
H2 tank pressure	37.4	bar	30	40
H2 flow rate	84.5	Nm³/h	50	150

Register address (decimal	Data point	Register value interpretation	Scaling factor	Engineering unit	Example from your screenshot
0	Electrolyzer temperature	Temperature × 10	Divide by 10	°C	65.0 °C
1	H2 tank pressure	Pressure × 10	Divide by 10	Bar	37.4 bar
2	H2 flow rate	Flow rate × 10	Divide by 10	Nm³/h	84.5 Nm³/h
3	Alarm status	Alarm indicator	No scaling identified	Status indicator	Not confirmed

Explanation of a normal Modbus communication

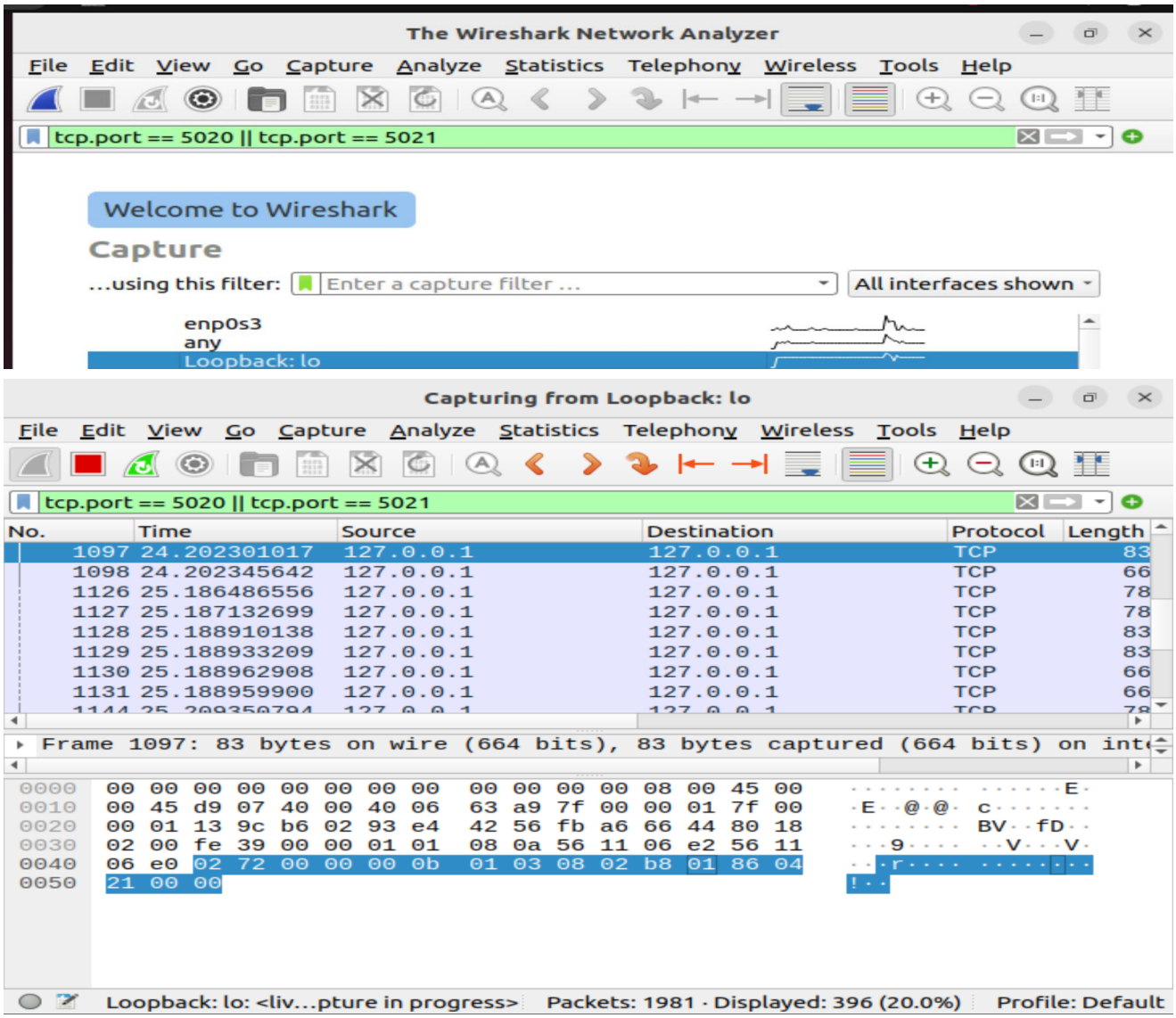
The client initiates a Modbus/TCP request to the local server. The request identifies the intended operation and relevant data address. The server processes the request and returns a response associated with that transaction. The returned value is interpreted using the register map and compared with the corresponding dashboard reading. This helps establish whether the observed protocol traffic is consistent with normal simulated process operation. The supplied OT simulation uses Modbus/TCP holding registers to represent process data for the hydrogen facility. The guarded dashboard reads four consecutive registers beginning at address 0, using a read request for four registers.

The first register represents electrolyzer temperature, the second represents hydrogen tank pressure, the third represents hydrogen flow rate, and the fourth represents alarm status. The dashboard divides the first three register values by 10 to convert them into their displayed engineering units. During the observed baseline, the dashboard displayed a temperature of 65.0 °C, tank pressure of 37.4 bar, and hydrogen flow rate of 84.5 Nm³/h. These values correspond to scaled register values of approximately 650, 374, and 845, respectively.

The dashboard's displayed operating ranges are 60-75 °C for temperature, 30-40 bar for tank pressure, and 50-150 Nm³/h for flow rate. All three observed process values were within their displayed ranges. The alarm register is also read by the dashboard, but its complete encoding requires confirmation from the plant implementation.

The register map provides a basis for interpreting Modbus/TCP traffic and comparing register values with the process dashboard. The register definitions and guard thresholds must be verified against the supplied source code.

Activity 03 - Packet visibility and protocol analysis



what does the capture prove? The capture provides evidence of Modbus/TCP communication on the Ubuntu guest's loopback interface during the recorded period. The matching request and response allow the client operation, server endpoint and relevant data fields to be examined. Where the returned register value agrees with the register map and dashboard, the capture also supports the interpretation of the simulated process state.

The capture does not prove that the guard prevents every unsafe operation, that clients are authenticated, that the service is secure against all attacks, or that the simulation is suitable for real industrial equipment.

Activity 04 - Safe discovery and exposure mapping

Local service inventory

Service	Endpoint	Function	Security concern
Direct plant	127.0.0.1:5020	Direct Modbus path	Direct access may bypass guard validation
Validation guard	127.0.0.1:5021	Validates and forwards accepted writes	Validation errors or incomplete rules
Direct dashboard	127.0.0.1:8080	Direct process view	Exposure of process information
Guarded dashboard	127.0.0.1:8081	Guarded process view	Exposure of process information

```
urity/sturity/student-lab-source/plant_sim$ ss -lntp
State      Recv-Q      Send-Q           Local Address:Port           Peer Address:P
ort      Process

LISTEN     0           128                  127.0.0.1:631                 0.0.0.0:*

LISTEN     0           100                  127.0.0.1:5020                0.0.0.0:*
         users:(("python3",pid=3581,fd=6))

LISTEN     0           100                  127.0.0.1:5021                0.0.0.0:*
         users:(("python3",pid=3609,fd=7))

LISTEN     0           4096             127.0.0.53%lo:53                  0.0.0.0:*

LISTEN     0           128                  127.0.0.1:8080                0.0.0.0:*
         users:(("python3",pid=3612,fd=4))

LISTEN     0           128                  127.0.0.1:8081                0.0.0.0:*
         users:(("python3",pid=3640,fd=4))

LISTEN     0           128                      [::1]:631                    [::]:*

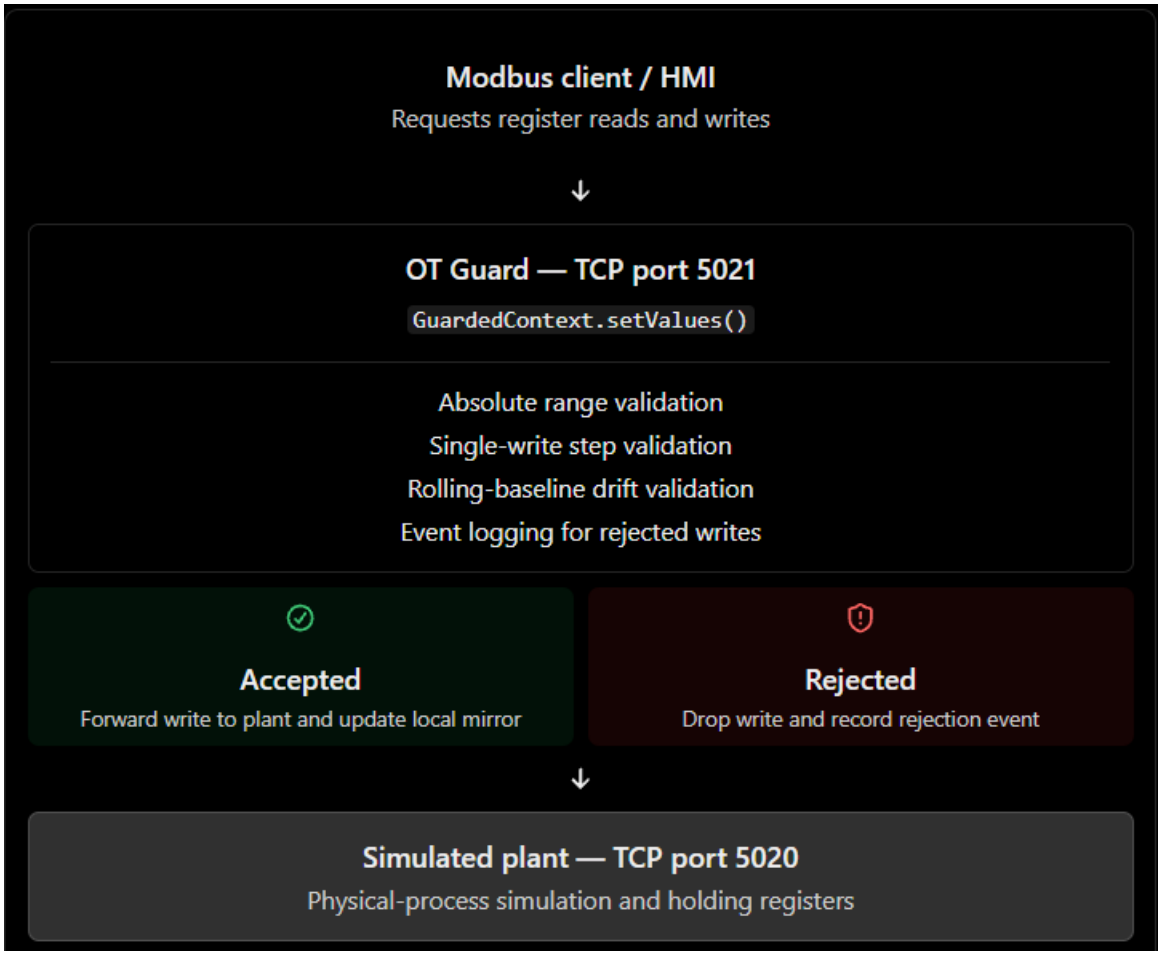
godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks/labs/ot-sec
```

Risk register

ID	Risk	Potential consequence	Suggested control	Evidence/status
R1	Direct-path access bypasses guard validation	Invalidated writes may alter simulated process values	Restrict access to the direct path and enforce validation at the trusted boundary	Confirm against source and test results
R2	Insufficient client authentication or authorization	An unauthorized client may be able to issue requests	Implement appropriate authentication, authorization and network access restrictions in real deployments	Verify what the simulator implements
R3	Incorrect or incomplete validation thresholds	Unsafe values may be accepted, or legitimate values may be rejected	Validate against process-specific engineering limits and test boundary cases	Analyze guard implementation
R4	Excessive requests affect service availability	Delayed or failed monitoring and control operations	Apply suitable rate limits, resource controls and monitoring	Do not infer a failure without evidence
R5	Process values are validated independently	Individually valid values may be inconsistent with one another	Use process-aware cross-variable validation where required	Establish whether correlated checks exist
R6	Insufficient audit evidence	Investigators may be unable to reconstruct a control decision	Maintain appropriate event logging, timestamps and review procedures	Inspect actual event records

Activity 05 - Analyze the supplied defensive OT guard

Guard architecture: The supplied guard acts as an intermediary between Modbus clients and the simulated industrial plant. It validates writes to protected telemetry registers, forwards accepted writes to the plant, and maintains a guarded copy of the plant state.



Key source components

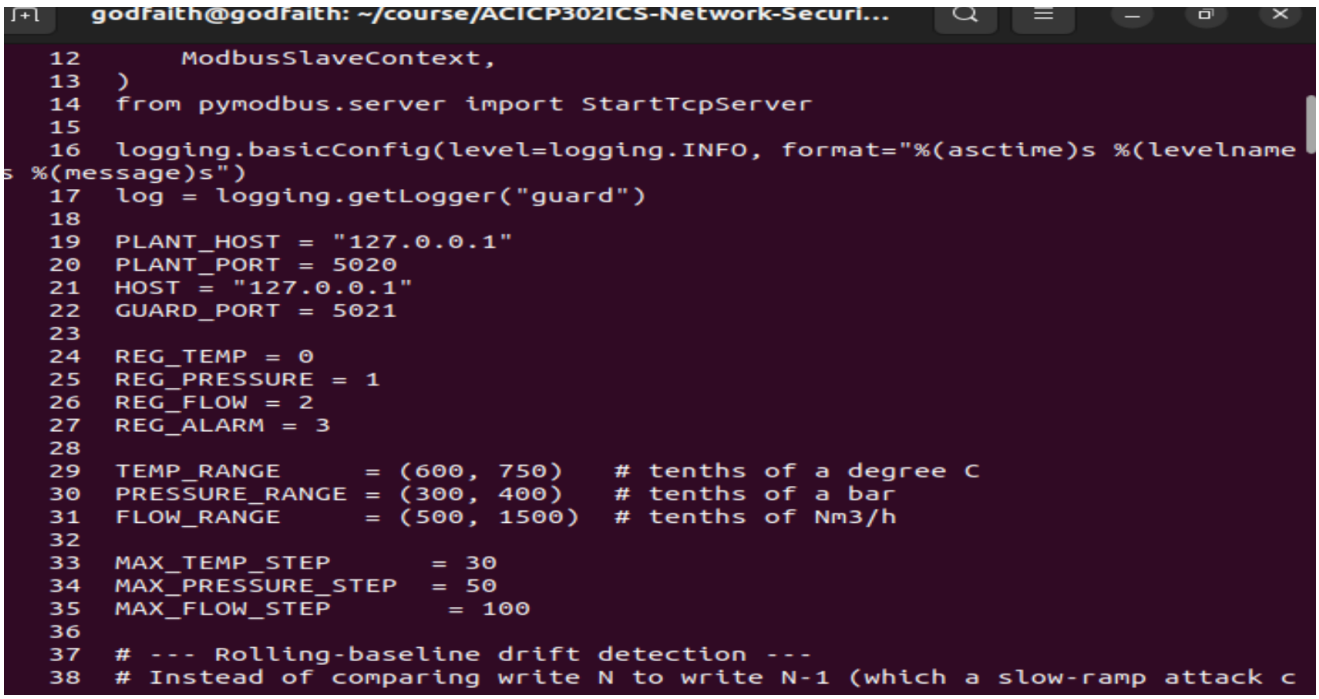
Component	Purpose
GuardedContext	Intercepts register writes and applies validation.
_check()	Evaluates range, step size and rolling-baseline drift.
setValues()	Rejects or accepts writes, forwards accepted writes, and logs rejections
sync_safe_reads()	Reads the plant periodically and mirrors telemetry when all three protected values are within range.
log_event()	Appends rejected-write events to events.jsonl
main()	Creates the Modbus client, register store, synchronization thread and TCP server.

Guard decision table

The following table documents the actual decision logic visible in the supplied source. The three telemetry checks are performed independently, and their rejection reasons are combined if more than one condition fails.

Check	Input	Condition	Decision/action	Audit Evidence
Absolute range	Requested temperature	Below 600 or above 750	Reject write	Python warning and JSONL event with outside safe range
Absolute range	Requested pressure	Below 300 or above 400	Reject write	Warning/event with outside safe range
Absolute range	Requested flow	Below 500 or above	Reject write	Warning/event with

		1500		outside safe range
Single-step limit	Requested and current accepted temperature	Absolute difference exceeds 30	Reject write	Warning/event with step of ... exceeds max
Single-step limit	Requested and current accepted pressure	Absolute difference exceeds 50	Reject write	Warning/event with step of ... exceeds max
Single-step limit	Requested and current accepted flow	Absolute difference exceeds 100	Reject write	Warning/event with step of ... exceeds max
Rolling-baseline drift	Requested value and oldest accepted value in a full three-write history	Absolute difference exceeds the variable's drift threshold	Reject write	Warning/event with cumulative drift ... over last 3 accepted writes exceeds ... limit
Accepted telemetry write	Any protected register	All applicable checks pass	Forward to plant and update local mirror; append accepted value to history	No acceptance event is explicitly written by this implementation
Unprotected register	Register other than 0, 1 or 2	Register enters the else branch	Attempt to forward write and update local store without telemetry validation	No rejection event is generated by this branch
Plant telemetry synchronization	Registers 0–2 read from plant	All three values are within their absolute safe ranges	Copy all four returned registers into the local mirror and append telemetry values to histories	No explicit synchronization event is logged
Unsafe plant telemetry	Registers 0–2 read from plant	Any protected value is outside its absolute range	Skip the entire mirror update	No explicit rejection event is logged for this condition



```
    41  # when every individual step looks small.
    42  DRIFT_WINDOW               = 3    # was 6 — shrinking cuts the attacker's
runway in half
    43  DRIFT_THRESHOLD_TEMP       = 20   # 2.0C cumulative over the window
    44  DRIFT_THRESHOLD_PRESSURE   = 20   # 2.0 bar cumulative over the window
    45  DRIFT_THRESHOLD_FLOW       = 40   # 4.0 Nm3/h cumulative over the window
    46
    47  EVENT_LOG_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__))
), "events.jsonl")
    48  event_log_lock = threading.Lock()
    49
    50
    51  def log_event(reason):
    52      event = {"timestamp": time.strftime("%Y-%m-%d %H:%M:%S"), "reason":
 reason}
    53      with event_log_lock:
    54          try:
    55              with open(EVENT_LOG_FILE, "a") as f:
    56                  f.write(json.dumps(event) + "\n")
    57          except Exception as e:
    58              log.error("Failed to write event log: %s", e)
    59
    60
    61  class GuardedContext(ModbusSlaveContext):
    62      """
    63      A Modbus write-through firewall.
    64
    65      Design rule: this class NEVER accepts a value it hasn't itself vali
dated.
```

```
godfaith@godfaith: ~/course/ACICP302ICS-Network-Securi...
   213      )
   214      sync_thread.start()
   215
   216      log.info("Guard listening on %s:%s — protecting plant at %s:%s",
   217               HOST, GUARD_PORT, PLANT_HOST, PLANT_PORT)
   218      try:
   219          StartTcpServer(context=context, address=(HOST, GUARD_PORT))
   220      except KeyboardInterrupt:
   221          pass
   222      finally:
   223          stop_event.set()
   224          plant_client.close()
   225
   226
   227  if __name__ == "__main__":
   228      main()
94:                f"{requested/unit_divisor:.1f}{unit_label} outside safe rang
e "
101:               f"step of {step/unit_divisor:.1f}{unit_label} exceeds max "
109:                   f"cumulative drift of {drift/unit_divisor:.1f}{unit_lab
el} "
141:                   log.error("Forward failed: %s", e)
147:           log.warning("BLOCKED write to reg %s: %s", address, reason_text
)
160:               log.error("Forward failed: %s", e)
196:               log.error("Sync error: %s", e)
```

Limitation Analysis: The supplied guard is a rule-based validation layer. Its effectiveness must be evaluated against the checks actually implemented in the source and the decisions observed during approved tests. A rejected write provides evidence that a particular rule operated under the tested conditions, but does not demonstrate comprehensive OT security. Authentication, authorization, direct-path bypass, availability, correlated process conditions and recovery remain separate concerns that must be assessed.

Authentication: validation of a value does not itself establish the identity of the client.

Authorization: a permitted value does not prove that the client is entitled to change that register.

Direct-path bypass: a client reaching the direct plant endpoint may bypass checks performed only by the guard.

Availability: validation does not automatically prevent service exhaustion or all denial-of-service conditions.

Correlated values: checking individual values may not establish whether temperature, pressure and flow are physically consistent.

Threshold tuning: limits must be appropriate to the process; overly broad or narrow limits may produce unsafe acceptance or false rejection.

Recovery: rejecting one request does not prove that the entire process has returned to a safe baseline.

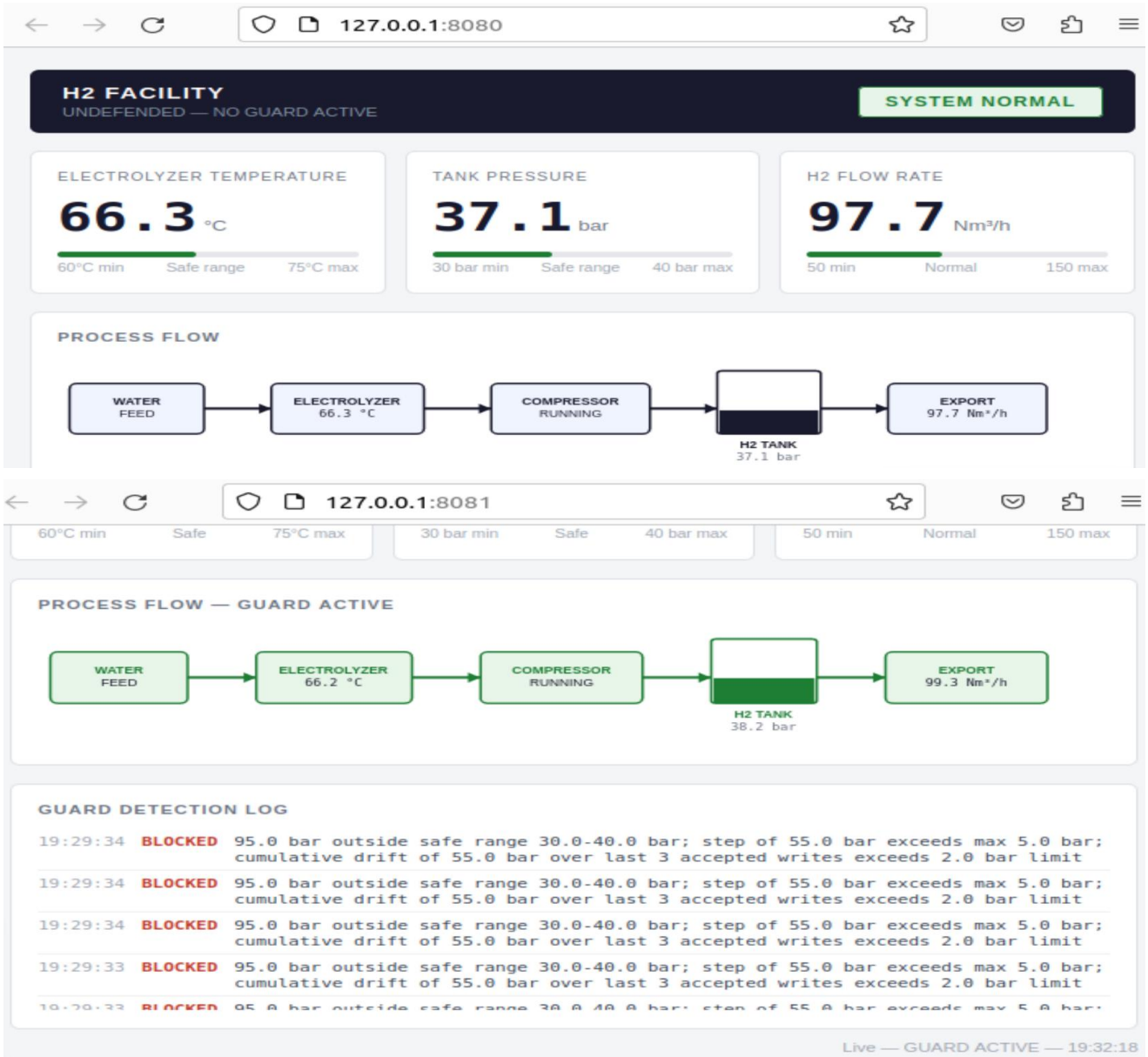
Evidence: Guard architecture/decision table; relevant source references; normal-pass/event sample; limitation analysis; updated risk entries.

Activity 06 - Controlled security validation

Compare direct and guarded behavior using approved bounded demonstrations supplied with the course source.

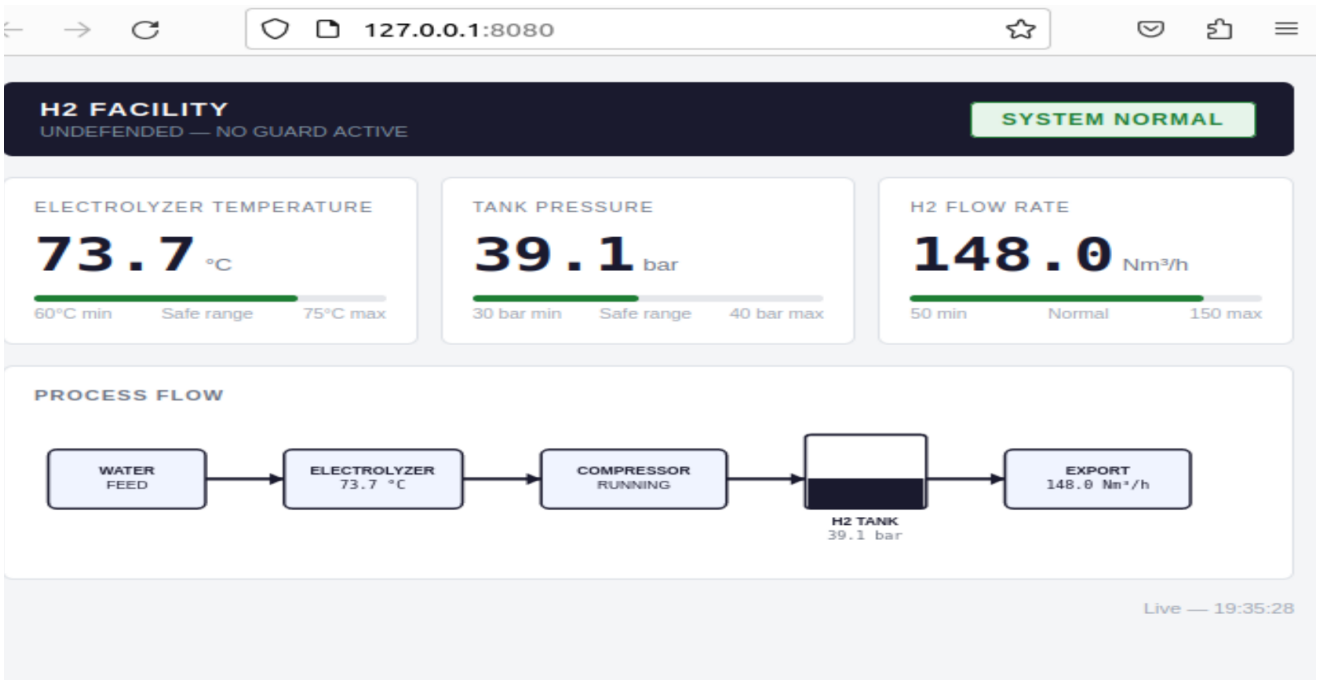
For Test A, python3 attack_hold.py

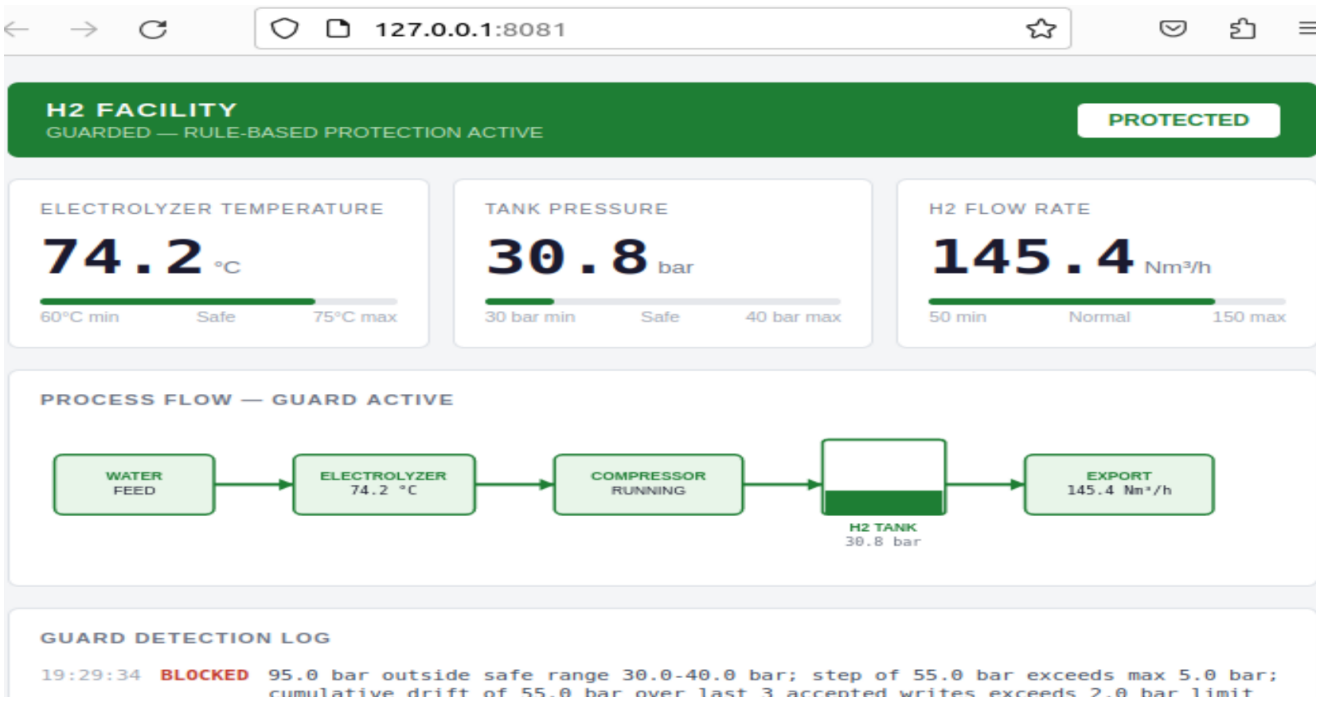
```
(.venv) godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ python3 attack_hold.py 5020
python3 attack_hold.py 5021
--- ATTACK HOLD: targeting port 5020 ---
Writes sent: 62
Peak pressure observed: 95.0 bar at t=0.03s (safe range: 30.0-40.0 bar)
Readings outside safe range: 62/62 (100%)
RESULT: UNSAFE VALUE REACHED THE PLANT
--- ATTACK HOLD: targeting port 5021 ---
Writes sent: 63
Peak pressure observed: 40.0 bar at t=0.27s (safe range: 30.0-40.0 bar)
Readings outside safe range: 0/63 (0%)
RESULT: value never left safe range — attack fully contained
(.venv) godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$
```



For Test B, python3 false_data_attack.py

```
(.venv) godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks/lab
s/ot-security/student-lab-source/plant_sim$ python3 false_data_attack.py 5020
python3 false_data_attack.py 5021
--- FALSE DATA INJECTION: port 5020 ---
Peak temp observed:      110.0C at t=0.01s (safe: 60.0-75.0C)
Peak pressure observed: 55.0bar at t=0.01s (safe: 30.0-40.0bar)
Peak flow observed:      200.0Nm3/h at t=0.01s (safe: 50.0-150.0)
Unsafe readings: temp 6/6, pressure 6/6, flow 6/6
RESULT: UNSAFE VALUES REACHED THE PLANT
--- FALSE DATA INJECTION: port 5021 ---
Peak temp observed:      75.0C at t=0.05s (safe: 60.0-75.0C)
Peak pressure observed: 40.0bar at t=0.05s (safe: 30.0-40.0bar)
Peak flow observed:      150.0Nm3/h at t=0.05s (safe: 50.0-150.0)
Unsafe readings: temp 0/6, pressure 0/6, flow 0/6
RESULT: attack fully contained
(.venv) godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks/lab
s/ot-security/student-lab-source/plant_sim$
```





For Test C, python3 slow_drift_attack.py

```
(.venv) godfaith@godfaith:~/course/ACICP302ICS-Network-Security-and-Attacks/lab
s/ot-security/student-lab-source/plant_sim$ python3 slow_drift_attack.py 5020
python3 slow_drift_attack.py 5021
Starting: 31.9 bar | Target: port 5020
  Step 1: requested 32.9 -> actual 32.8
  Step 2: requested 33.9 -> actual 33.7
  Step 3: requested 34.9 -> actual 35.1
  Step 4: requested 35.9 -> actual 36.1
  Step 5: requested 36.9 -> actual 37.3
  Step 6: requested 37.9 -> actual 37.9
  Step 7: requested 38.9 -> actual 39.3
  Step 8: requested 39.9 -> actual 40.0
  Step 9: requested 40.9 -> actual 40.0
  Step 10: requested 41.9 -> actual 40.0
  Step 11: requested 42.9 -> actual 39.9
  Step 12: requested 43.9 -> actual 39.8
Starting: 39.9 bar | Target: port 5021
  Step 1: requested 40.9 -> actual 39.7
  Step 2: requested 41.9 -> actual 39.4
  Step 3: requested 42.9 -> actual 39.1
  Step 4: requested 43.9 -> actual 39.0
  Step 5: requested 44.9 -> actual 39.2
  Step 6: requested 45.9 -> actual 39.0
  Step 7: requested 46.9 -> actual 39.0
  Step 8: requested 47.9 -> actual 39.3
  Step 9: requested 48.9 -> actual 39.6
```

Test Results and Comparison

Three controlled demonstrations were conducted to compare the direct Modbus/TCP path on port 5020 with the guarded path on port 5021.

Test A: Pressure Hold. On the direct path, 62 writes were sent, the peak observed pressure reached 95.0 bar, and all 62 readings were outside the 30-40 bar safe range. On the guarded path, the peak pressure was 40.0 bar, with zero unsafe readings out of 63. This indicates that the guard contained the tested pressure writes within the specified range.

Test B: False Data Injection. On the direct path, the observed peaks were 110.0 °C for temperature, 55.0 bar for pressure, and 200.0 Nm^3/h for flow. All six readings for each variable were classified as unsafe. On the guarded path, the observed peaks were 75.0 °C, 40.0 bar, and 150.0 Nm^3/h, respectively, with zero unsafe readings for all three variables. This supports the conclusion that the guard's validation rules contained the tested out-of-range values.

Test C: Slow Drift. On the direct path, pressure started at 31.9 bar and rose as the requested values increased, reaching 40.0 bar. On the guarded path, pressure started at 39.9 bar, and the visible results show actual pressure remaining approximately between 39.0 and 39.7 bar despite increasing requested values. The guarded test output must be reviewed to completion before a definitive conclusion is made about the final outcome.

Overall, the first two demonstrations show that the guard effectively constrained the tested out-of-range pressure and false-data writes. The visible slow-drift results suggest that gradual changes were also constrained, but the complete output is needed to confirm the final behavior. These findings apply to the supplied simulation and the specific test conditions; they do not establish comprehensive OT security.